

Adding Custom IP to an Embedded System Using AXI

Introduction

This lab guides you through the process of adding a custom peripheral to a processor system by using the Create and Import Peripheral Wizard.

Objectives

After completing this lab, you will be able to:

- Create a custom peripheral
- Add the custom peripheral to your design
- Add pin location constraints
- Generate the hardware bitstream

Procedure

This lab is separated into steps that consist of general overview statements that provide information on the detailed instructions that follow. Follow these detailed instructions to progress through the lab.

This lab comprises 4 primary steps: You will open the project, generate a peripheral template, create a peripheral, and, finally, add and connect the peripheral.

Design Description

You will extend the Lab 2 hardware design by creating and adding a AXI peripheral (refer to MYIP in **Figure 1**) to the system, and connecting it to the LEDs on the Nexys4DDR Board. You will use the IP Packager to generate the custom IP. Next, you will connect the peripheral to the system and add pin location constraints to connect the LED display controller peripheral to the on-board LED display. Finally, you will add BRAM Controller and BRAM before generating the bitstream.

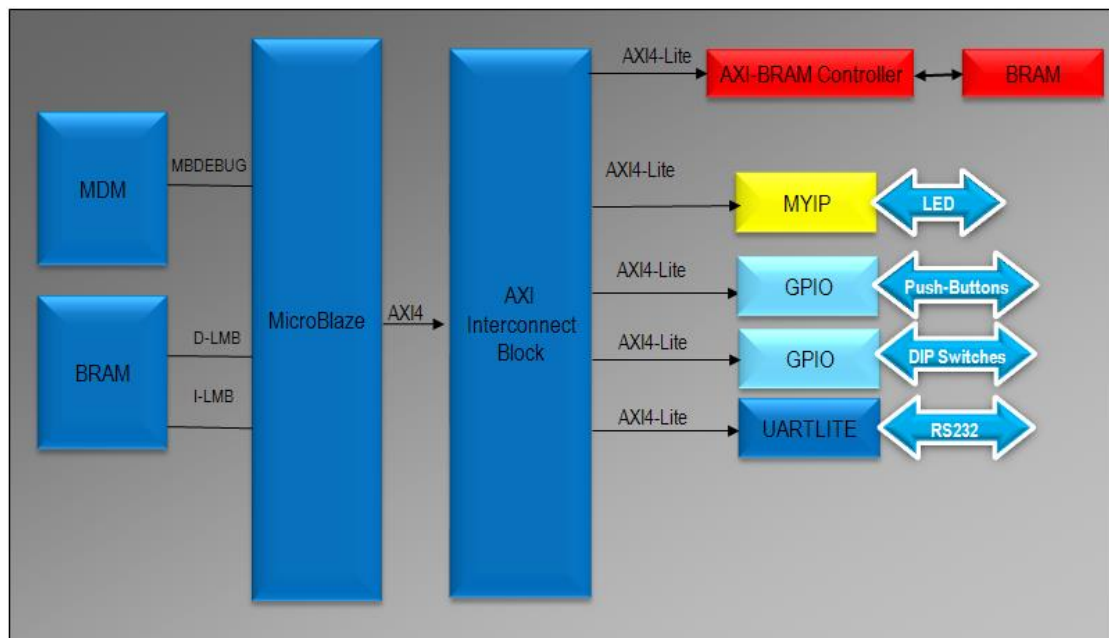
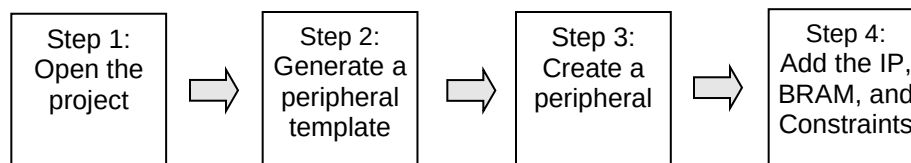


Figure 1. Design updated from previous lab

General Flow for this Lab



In the instructions below:

{sources} refers to `labs_microblaze.src` (eKursy),

{labs} refers to local user directory

Create a Custom IP using the Create and Package IP Wizard Step 1

1-1. Use the provided `axi_lite` slave peripheral template and the custom IP source code to create a custom IP.

1-1-1. Open Vivado by selecting **Start > All Programs > Xilinx Design Tools > Vivado 2024**

1-1-2. Click **Tasks > Manage IP** and select *New IP Location...* and click **Next** in the *New IP Location* window

1-1-3. Select **VHDL** as the *Target Language*, **Vivado** as *Target simulator*, **Mixed** as the *Simulator language*, and for *IP location*, type `{labs}\led_ip` and click **Finish** (leave other settings as defaults and click **OK** if prompted to create the directory)

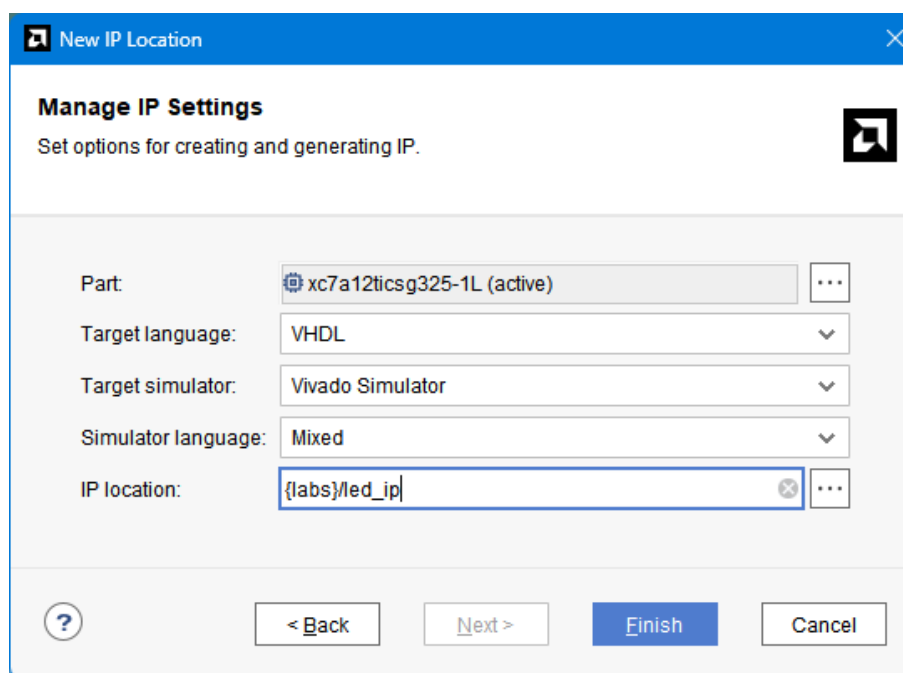
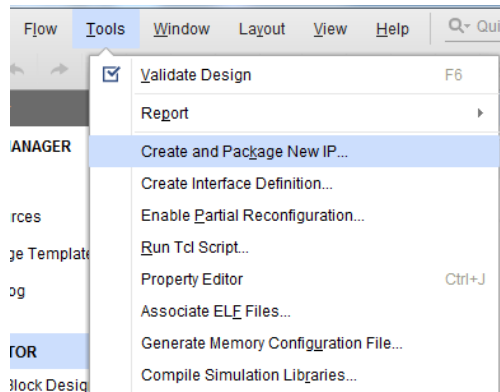


Figure 2. New IP Location form

A Kintex 7 part is chosen for this project, but later compatibility for other devices will be added to the packaged IP.

1-2. Run the Create and Package IP Wizard

1-2-1. Select *Tools > Create and Package New IP*



1-2-2. In the window, click **Next**

1-2-3. Select *Create a new AXI4 peripheral*, and click **Next**

1-2-4. Fill in the details for the IP

Name: **led_ip**

Display Name: **led_ip_v1_0**

(Fill in a description, Vendor Name, URL ...)

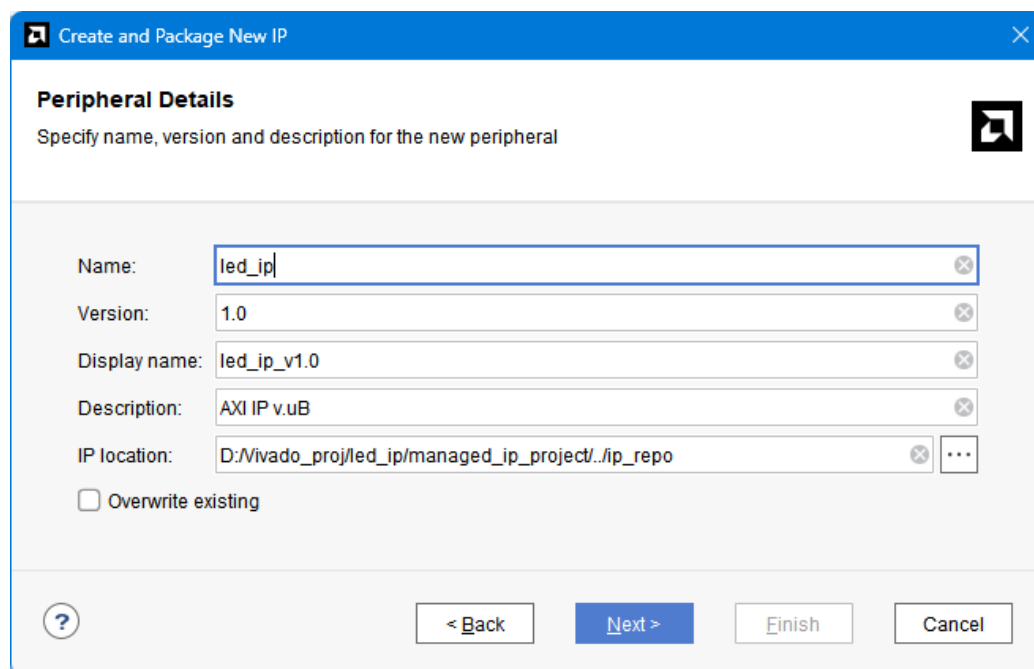
A screenshot of the 'Create and Package New IP' dialog box in Vivado. The dialog has a blue title bar and a white body. The 'Peripheral Details' section is active, with the instruction 'Specify name, version and description for the new peripheral'. There are five text input fields: 'Name' (containing 'led_ip'), 'Version' (containing '1.0'), 'Display name' (containing 'led_ip_v1.0'), 'Description' (containing 'AXI IP v.uB'), and 'IP location' (containing 'D:/vivado_proj/led_ip/managed_ip_project/..ip_repo'). Each field has a small 'x' icon to its right. Below the fields is an unchecked checkbox labeled 'Overwrite existing'. At the bottom of the dialog are four buttons: a help button (question mark icon), '< Back', 'Next >', and 'Cancel'. The 'Next >' button is highlighted in blue.

Figure 3. Updating Peripheral Details form

1-2-5. Click **Next**

1-2-6. Change the Name of the interface to **S_AXI**

1-2-7. Leave the other settings as default and click **Next** (Lite interface, Slave mode, Data Width 32, Registers 4)

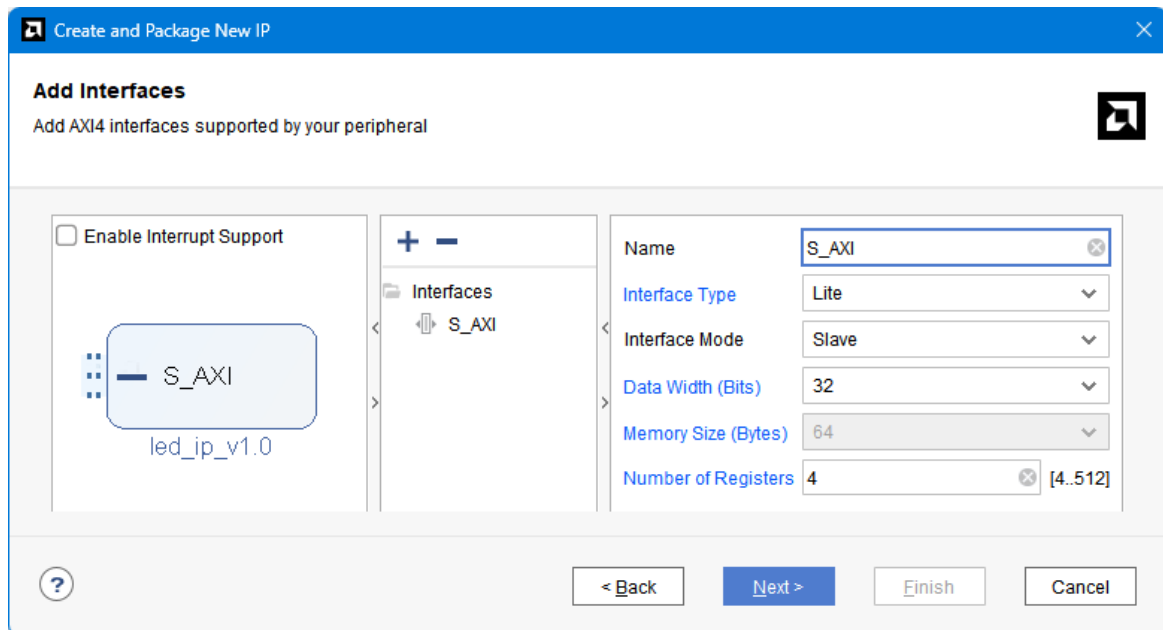
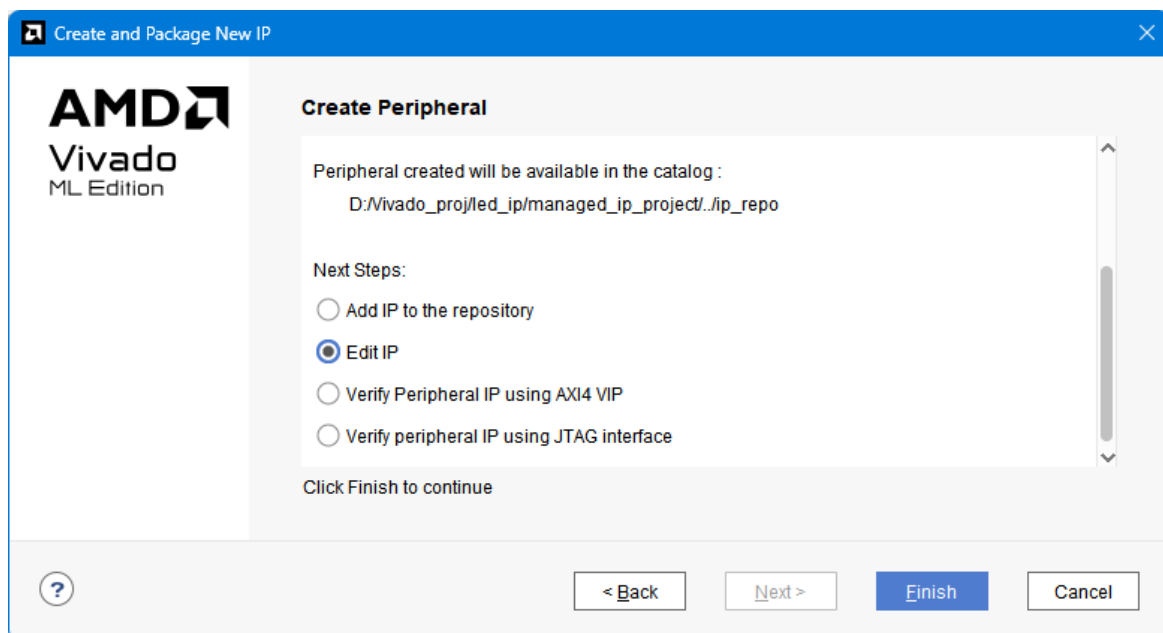


Figure 4. Naming the AXI interface

1-2-8. Select *Edit IP* and click **Finish** (a new Vivado Project will open)



1-3. Create an interface to the LEDs

1-3-1. In the sources panel, double-click the **led_ip.vhd** file.

This file contains the HDL code for the interface(s) selected above. The top level file contains a module which implements the AXI interfacing logic, and an example design to write to and read from the number of registers specified above. This template can be used as a basis for creating custom IP. A new parameterized output port to the LEDs will be created at the top level of the design, and the AXI write data in the sub-module will be connected back up to the external LED port.

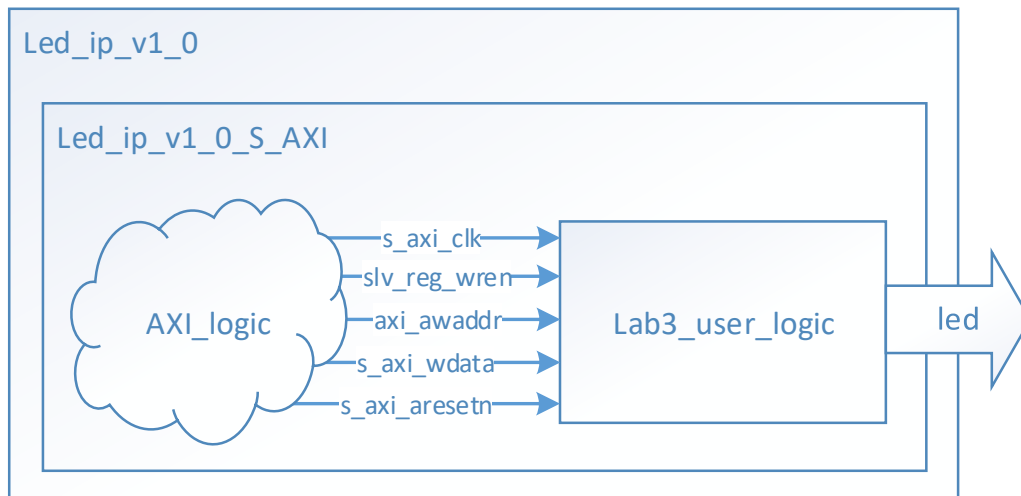


Figure 5. Custom IP architecture

Scroll down to line 7 where a user *parameters* space is provided.

1-3-2. Add the line:

LED_WIDTH : integer = 8;

Go to line 18 and add the line:

led : out std_logic_vector(LED_WIDTH-1 downto 0);

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity led_ip is
6      generic (
7          -- Users to add parameters here
8          LED_WIDTH : integer := 8;
9          -- User parameters ends
10         -- Do not modify the parameters beyond this line
11
12
13         -- Parameters of Axi Slave Bus Interface S_AXI
14         C_S_AXI_DATA_WIDTH : integer := 32;
15         C_S_AXI_ADDR_WIDTH : integer := 4
16     );
17     port (
18         -- Users to add ports here
19         led : out std_logic_vector(LED_WIDTH-1 downto 0);
20         -- User ports ends
21         -- Do not modify the ports beyond this line

```

Figure 6. Adding users generic parameter, and port definition

1-3-3. Fill in generic and port definition in lower-level component declaration (line 53 and 58)

```

49 architecture arch_imp of led_ip is
50
51     -- component declaration
52     component led_ip_slave_lite_v1_0_S_AXI is
53         generic (
54             LED_WIDTH : integer := 8;
55             C_S_AXI_DATA_WIDTH : integer := 32;
56             C_S_AXI_ADDR_WIDTH : integer := 4
57         );
58         port (
59             led : out std_logic_vector(LED_WIDTH-1 downto 0);
60             S_AXI_ACLK : in std_logic;
61             S_AXI_ARESETN : in std_logic;
62             S_AXI_AWADDR : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
63             S_AXI_AWPROT : in std_logic_vector(2 downto 0);

```

1-3-4. Insert the following at line ~88:

```
LED_WIDTH => LED_WIDTH,
```

1-3-5. Insert the following at line ~93:

```
LED => LED,
```

```

86 -- Instantiation of Axi Bus Interface S_AXI
87 led_ip_slave_lite_v1_0_S_AXI_inst : led_ip_slave_lite_v1_0_S_AXI
88     generic map (
89         LED_WIDTH => LED_WIDTH,
90         C_S_AXI_DATA_WIDTH => C_S_AXI_DATA_WIDTH,
91         C_S_AXI_ADDR_WIDTH => C_S_AXI_ADDR_WIDTH
92     )
93     port map (
94         led => led,
95         S_AXI_ACLK => s_axi_aclk,
96         S_AXI_ARESETN => s_axi_aresetn,
97         S_AXI_AWADDR => s_axi_awaddr,
98         S_AXI_AWPROT => s_axi_awprot,

```

Figure 7. Adding port connection with a lower-level module

1-3-6. Save the file by selecting **File > Save File**

1-3-7. Expand **led_ip** in the sources view if necessary, and open **led_ip_slave_lite_v1_0_S_AXI.vhd**

1-3-8. Edit the file according to custom IP architecture (Figure.5)

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity led_ip_slave_lite_v1_0_S_AXI is
6      generic (
7          -- Users to add parameters here
8          LED_WIDTH : integer := 8;
9          -- User parameters ends
10         -- Do not modify the parameters beyond this line
11
12         -- Width of S_AXI data bus
13         C_S_AXI_DATA_WIDTH : integer := 32;
14         -- Width of S_AXI address bus
15         C_S_AXI_ADDR_WIDTH : integer := 4
16     );
17     port (
18         -- Users to add ports here
19         led : out std_logic_vector(LED_WIDTH-1 downto 0);
20         -- User ports ends

```

Figure 8. Declaring users port in the lower-level module for the Nexys4DDR

1-3-9. Scroll down to **~line 320** and insert the following code to instantiate the user logic for the LED IP

```

324     -- Add user logic here
325     user_logic: lab3_user_logic
326     generic map(LED_WIDTH=>LED_WIDTH)
327     port map(
328         S_AXI_ACLK => S_AXI_ACLK,
329         slv_reg_wren => axi_awready,
330         axi_awaddr => axi_awaddr(C_S_AXI_ADDR_WIDTH-1 downto ADDR_LSB),
331         s_axi_wdata => s_axi_wdata,
332         s_axi_aresetn => s_axi_aresetn,
333         led => led
334     );
335     -- User logic ends
336
337 end arch_imp;

```

Figure 9. Instantiating lower-level user module

Save the file and check all the signals that are being connected and where they originate.

1-3-10. Download the **lab3_user_logic.vhd** file from {[sources](#)}

1-3-11. Click on the **Add Sources** in the Flow Navigator pane, select **Add or Create Design Sources**, click **Next**, then click the **Blue Plus** then **Add Files...**, select the **lab3_user_logic.vhd** file and click **OK**, and then click **Finish** to add the file.

Check the contents of this file to understand the logic that is being implemented. Notice the formed hierarchy.

1-3-12. Click **Run Synthesis** and **Save** if prompted. (This is to check the design synthesizes correctly before packaging the IP. **If this was your own design, you would simulate it and verify functionality before proceeding**)

1-3-13. **Check the Messages tab for any errors and correct if necessary before moving to the next step**

When Synthesis completes successfully, click **Cancel**.

1-4. Package the IP

1-4-1. Click on the **Package IP – led_ip** tab

The screenshot shows the 'Package IP - led_ip' tab in the Vivado IP Packager tool. The 'Packaging Steps' sidebar on the left lists several steps: Identification (checked), Compatibility (checked), File Groups, Customization Parameters, Ports and Interfaces, Addressing and Memory (checked), Customization GUI, and Review and Package. The main area displays the 'Identification' section with the following fields:

- Vendor: CCE
- Library: CCE
- Name: led_ip
- Version: 1.0
- Display name: led_ip_v1.0
- Description: AXI IP v.uB
- Vendor display name: {Vendor Name}
- Company url:
- Root directory: d:/vivado_proj/led_ip/ip_repo/led_ip_1_0
- Xml file name: d:/vivado_proj/led_ip/ip_repo/led_ip_1_0/component.xml

Below the Identification section is the 'Categories' section, which includes a list of categories. The 'AXI_Peripheral' category is currently selected.

Figure 10. Package IP

1-4-2. For the IP to appear in the IP catalog in particular categories, the IP must be configured to be part of those categories. To change which categories the IP will appear in the IP catalog click **Blue Plus** in the *Categories* section. This opens the Choose IP Categories window

1-4-3. For the purpose of this exercise, uncheck the **AXI Peripheral** box and check the **Basic Elements** and click **OK**.

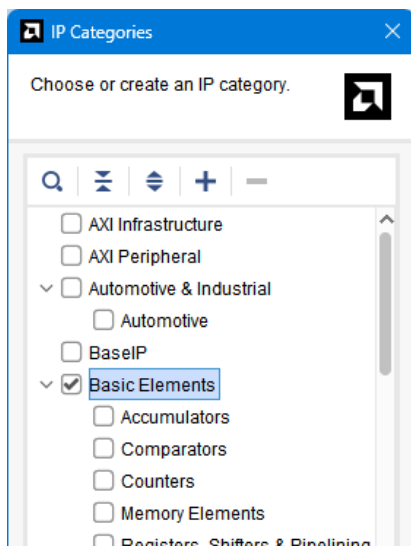


Figure 11. Specify the category for IP Packager IP

- 1-4-4.** Select **Compatibility**. This shows the different Xilinx FPGA Families that the IP supports. The value is inherited from the device selected for the project.
- 1-4-5.** Click the **Blue Plus** then **Add Family Explicitly...** from the menu.
- 1-4-6.** Select the **Artix-7** family as we will be using this IP on the Nexys4DDR board, and click **OK**.
- 1-4-7.** You can also customize the address space and add memory address space using the **IP Addressing and Memory** category. We won't make any changes.
- 1-4-8.** Click on **File Groups** and click [Merge changes from File Groups Wizard](#)

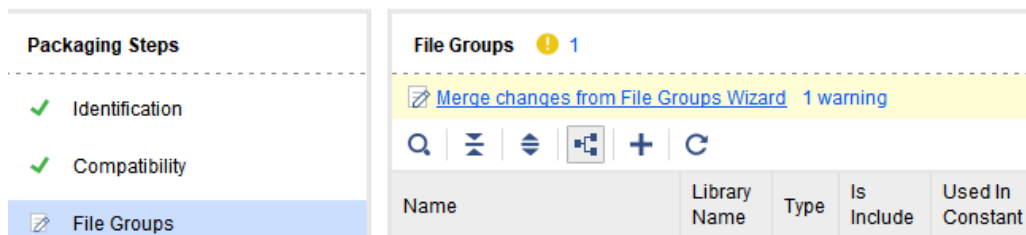
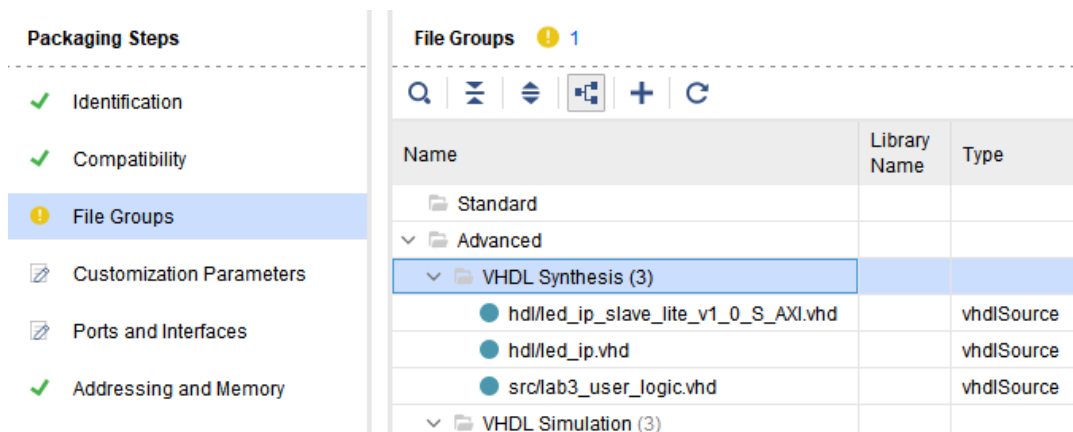


Figure 12. Updating the file group

This is to update the IP Packager with the changes that were made to the IP and the `lab3_user_logic_*.vhd` file that was added to the project.

- 1-4-9.** Expand **VHDL Synthesis** and notice **lab3_user_logic.vhd** has been included



- 1-4-10.** Click on **Customization Parameters** and again [Merge changes from Customization Parameters Wizard](#)

Name	Description	Display Name	Value	Value Bit St Length
Customization Parameters				
C_S_AXI_DATA_WIDTH	Width of S_AXI data bus	C S AXI DATA WIDTH	32	0
C_S_AXI_ADDR_WIDTH	Width of S_AXI address bus	C S AXI ADDR WIDTH	4	0
C_S_AXI_BASEADDR		C S AXI BASEADDR	0xFFFFFFFF	32
C_S_AXI_HIGHADDR		C S AXI HIGHADDR	0x00000000	32
Hidden Parameters				
LED_WIDTH		Led Width	8	0

Figure 13. User parameter

1-4-11. Select **Customization Parameters**, expand *Hidden Parameters*, right-click on **LED_WIDTH**, and select *Import IP Parameters...* and click **OK**.

1-4-12. Select **Customization GUI** and notice that the *Led Width* is visible.

Layout

- Window
 - Component Name
 - Page 0
 - C S AXI BASEADDR
 - C S AXI HIGHADDR
 - Led Width
 - C S Axi Data Width
 - C S Axi Addr Width
 - Hidden Parameters

Preview

☐ Show disabled ports

Component Name: led_ip_0

C S AXI BASEADDR: 0xFFFFFFFF

C S AXI HIGHADDR: 0x00000000

Led Width: 8

C S Axi Data Width: 32

Figure 14. User customization parameter

1-4-13. Select *Review and Package*, and notice the path where the IP will be created.

Packaging Steps

- ✓ Identification
- ✓ Compatibility
- ! File Groups
- ✓ Customization Parameters
- ! Ports and Interfaces
- ✓ Addressing and Memory
- ✓ Customization GUI
- Review and Package**

Review and Package

! 2 warnings 1 info message

Summary

Display name: led_ip_v1.0

Description: AXI IP v.uB

Root directory: d:/Vivado_proj/led_ip/ip_repo/led_ip_1_0

After Packaging

An archive will not be generated. Use the settings link below to change your preference

Project will be removed after completion

[Edit packaging settings](#)

Re-Package IP

Verify packaging settings:

IP > Packager
Specify settings related to IP Packager.

Default Values

The following values will be automatically applied after finishing the IP Packager Wizard.

Vendor: CCE

Library: CCE

Category: CCE_LIB

IP location: ../ip_repo

Automatic Behavior

After Packaging

- ☒ Create archive of IP
- ☒ Add IP to the IP Catalog of the current project
- ☒ Close IP Packager window
- ☐ Include Source project archive

Edit IP in IP Packager

- ☐ Delete project after packaging

1-4-14. Click **Re-Package IP**. Click **Yes** and the project will close when complete.

1-4-15. In the original Vivado window click **File > Project -> Close**

Modify the Project Settings

Step 3

2-1. Open the previous project, or use the lab2 project from the {labs} directory, and save as project_3. Set Project Settings to point to the created IP repository.

2-1-1. Start the Vivado if necessary and open the project_2 you created in the previous lab

2-1-2. Save design as a new project (Tcl Console):
`save_project_as project_3 {labs}/project_3 -force`

2-1-3. Click **Project Settings** in the *Flow Navigator* pane.

2-1-4. Select **IP** in the left pane of the *Project Settings* form.

2-1-5. Click on the **Blue Plus** button, browse to {labs}\led_ip and click **Select**.

The led_ip_v1.0 IP will appear the **IP in the Selected Repository** window.

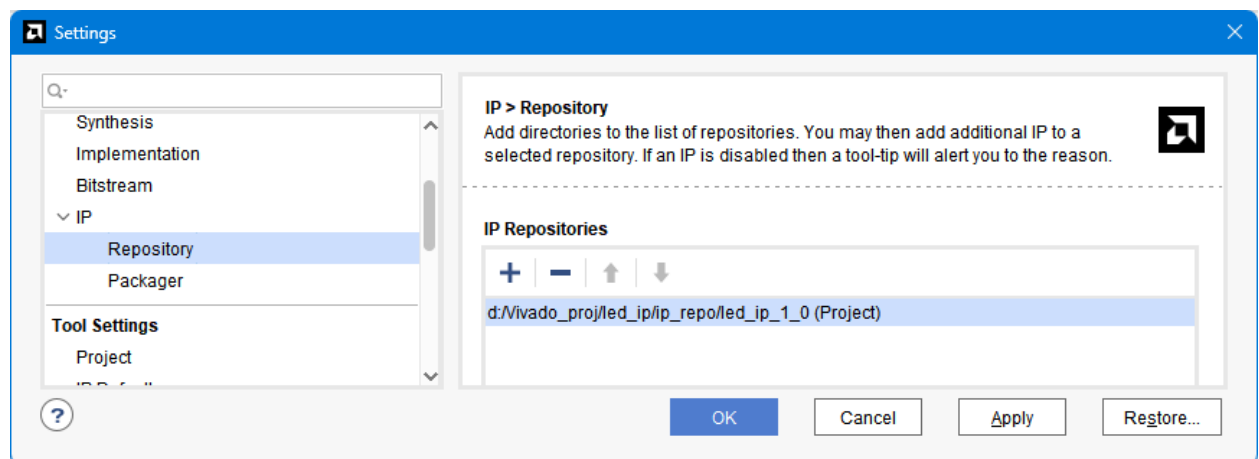


Figure 15. Specify IP Repository

2-1-6. Click **OK**.

Add the Custom IP, BRAM, and the Constraints

Step 4

3-1. Add led_ip to the design and connect to the AXI4Lite interconnect in the IPI. Make internal and external port connections. Establish the LED port as external FPGA pins.

3-1-1. Click **Open Block Design** under **IP Integrator** in the *Flow Navigator* pane

- 3-1-2.** Click the Add IP icon (+) and search for **led_ip_v1.0** in the catalog by typing “led” in the search field.

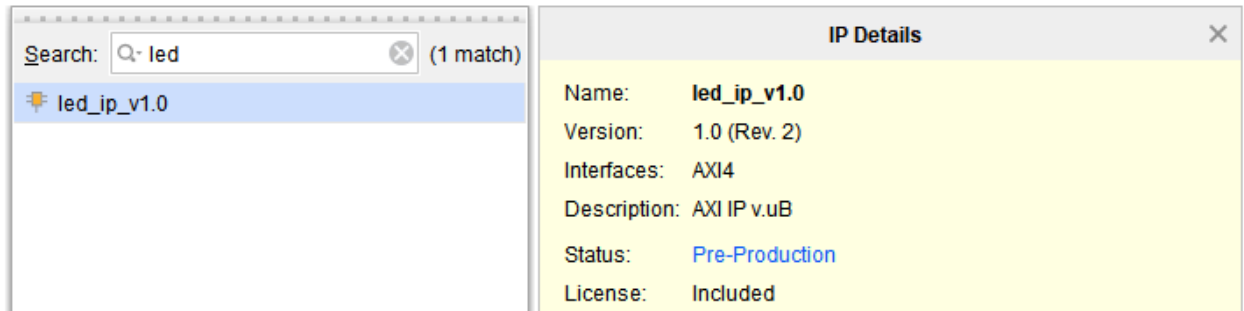


Figure 16. Searching for led_ip in the IP Catalog

- 3-1-3.** Double-click **led_ip_v1_0** to add the core to the design.
- 3-1-4.** Select the IP in the block diagram and change the instance name to **led_ip** in the properties view.
- 3-1-5.** Double click the block to open the configuration properties
- 3-1-6.** For the **Nexys4DDR** board, set the width to **16**.
- 3-1-7.** Click **OK**.

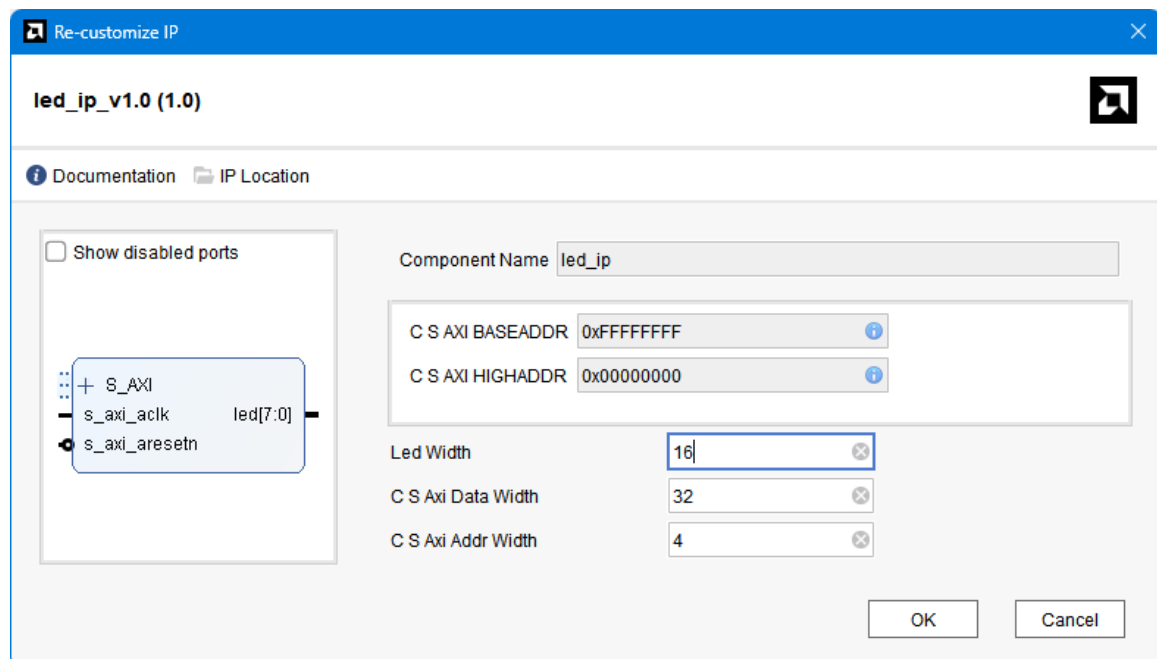


Figure 17. Configure the LED IP LED_WIDTH

Click on **Run Connection Automation**, select **/led_ip/S_AXI** and click **OK** to automatically make the connection from the AXI Interconnect to the IP.

Click the regenerate button () to redraw the diagram.

- 3-1-8.** Select the *LED* port on the *led_ip* instance (by clicking on its pin), right-click and select **Make External**.

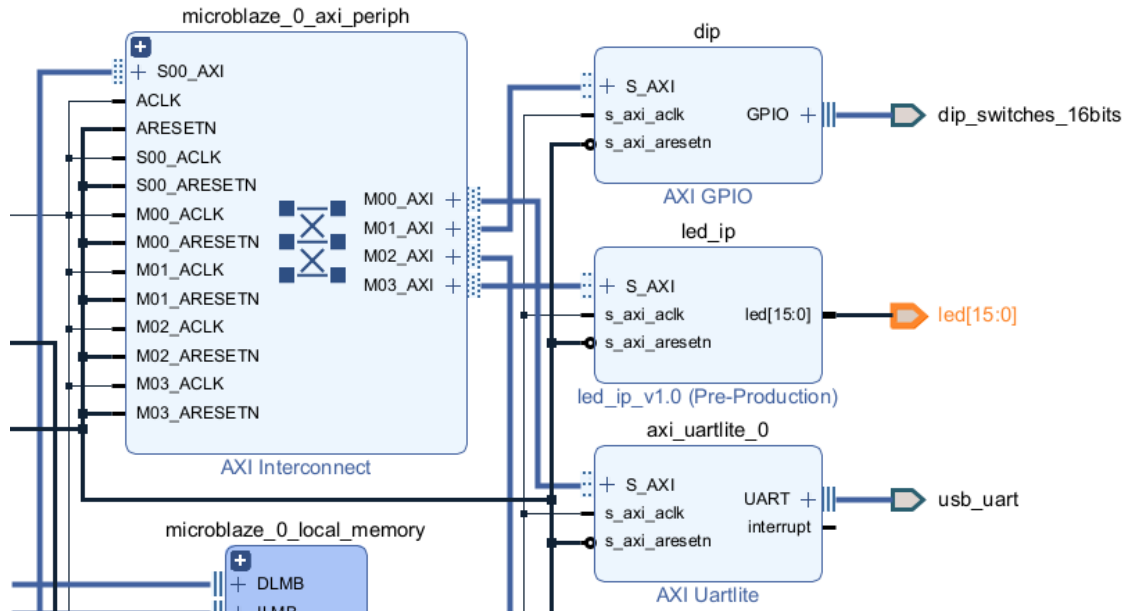


Figure 18. LED external port added and connected

- 3-1-9.** Select the **Address Editor** tab and verify that an address has been assigned to *led_ip*.

Diagram x Address Editor x Address Map x						
☒ Assigned (6) ☒ Unassigned (0) ☒ Excluded (0) Hide All						
Name	Interface	Slave Segment	Master Base Address	Range	Master High Address	
Network 0						
/microblaze_0						
/microblaze_0/Data (32 address bits : 4G)						
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF	
/dip/S_AXI	S_AXI	Reg	0x4000_0000	64K	0x4000_FFFF	
/led_ip/S_AXI	S_AXI	S_AXI_reg	0x44A0_0000	64K	0x44A0_FFFF	
/microblaze_0_local_memory/dlmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	16K	0x3FFF	
/push/S_AXI	S_AXI	Reg	0x4001_0000	64K	0x4001_FFFF	
Network 1						
/microblaze_0						
/microblaze_0/Instruction (32 address bits : 4G)						
/microblaze_0_local_memory/ilm_bram_if_cntlr/SLMB	SLMB	Mem	0x0	16K	0x3FFF	

Figure 19. Address assigned for led_ip

3-2. Add BRAM to the design

- 3-2-1. In the Block Diagram, click the Add IP icon (+) and search for BRAM and add one instance of the **AXI BRAM Controller**
- 3-2-2. Run *Connection Automation* on **axi_bram_ctrl_0/S_AXI** and click **OK** when prompted to connect it to the **M_AXI_GP0 Master**.
- 3-2-3. Double click on the block to customize it and change the number of BRAM interfaces to **1** and click **OK**.

Notice that the AXI Protocol being used is AXI4 instead of AXI4Lite since BRAM can provide higher bandwidth and the controller can support burst transactions.

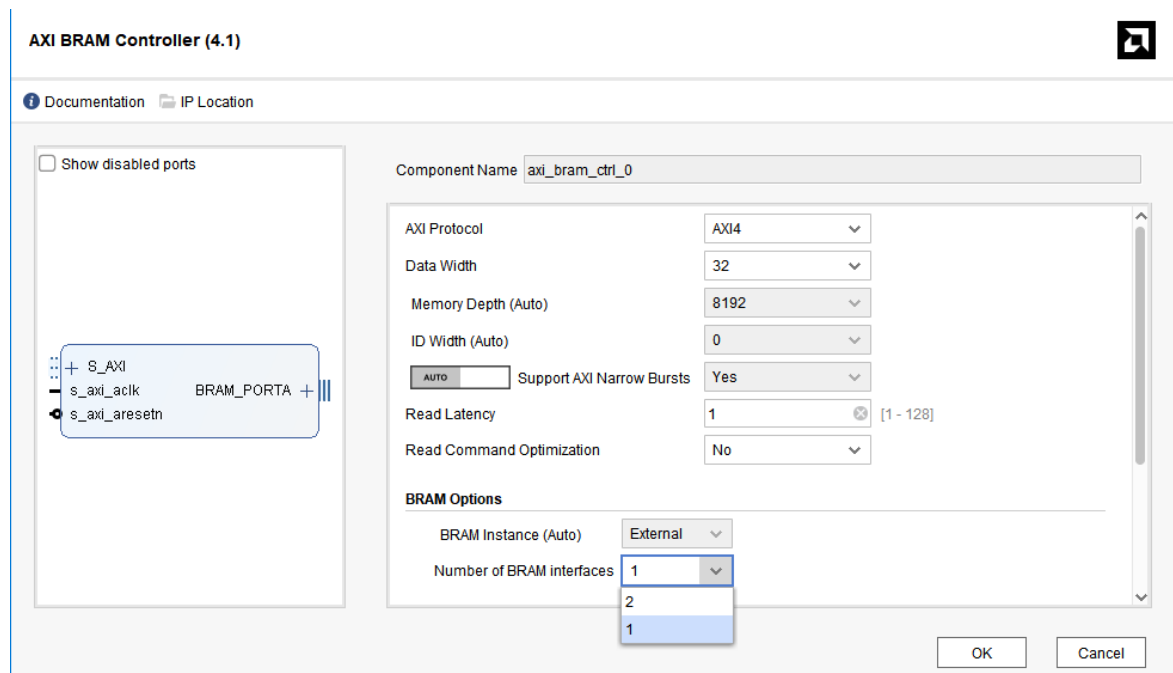
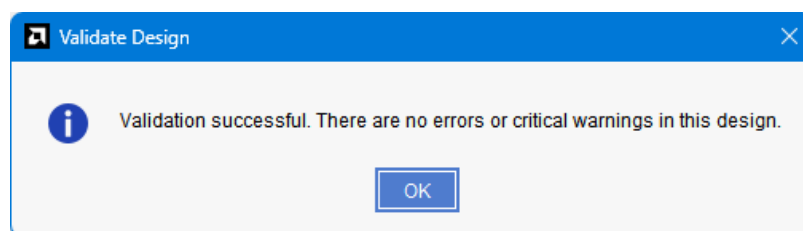


Figure 20. Customize BRAM controller

- 3-2-4. Click on *Run Connection Automation* to add and connect a **Block Memory Generator** by selecting **axi_bram_ctrl_0/BRAM_PORTA** and click **OK** (This could be added manually)
- 3-2-5. Validate the design to ensure there are no errors (F6), and click the regenerate button () to redraw the diagram.



The design should look similar to the figure below.

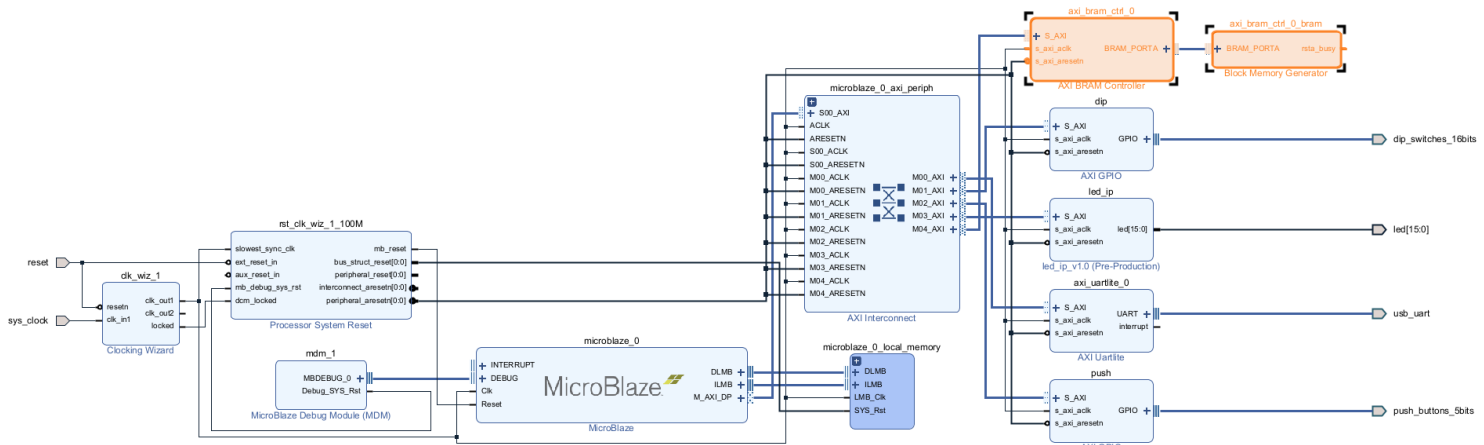


Figure 21. Completed Block Diagram

3-2-6. In the Address editor, notice the Range of the `axi_bram_ctrl_0` is 8K. We will leave it at that.

Address Editor						
Address Map						
Diagram						
☒ Assigned (7) ☒ Unassigned (0) ☒ Excluded (0) Hide All						
Name	Interface	Slave Segment	Master Base Address	Range	Master High Add	
Network 0						
/microblaze_0						
/microblaze_0/Data (32 address bits : 4G)						
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem0	0xC000_0000	8K	0xC000_1FFF	
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF	
/dip/S_AXI	S_AXI	Reg	0x4000_0000	64K	0x4000_FFFF	
/led_ip/S_AXI	S_AXI	S_AXI_reg	0x44A0_0000	64K	0x44A0_FFFF	
/microblaze_0_local_memory/dlmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	16K	0x3FFF	
/push/S_AXI	S_AXI	Reg	0x4001_0000	64K	0x4001_FFFF	
Network 1						
/microblaze_0						
/microblaze_0/Instruction (32 address bits : 4G)						
/microblaze_0_local_memory/ilmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	16K	0x3FFF	

Figure 22. Adjusting memory size

3-2-7. Press **F6** to validate the design one last time.

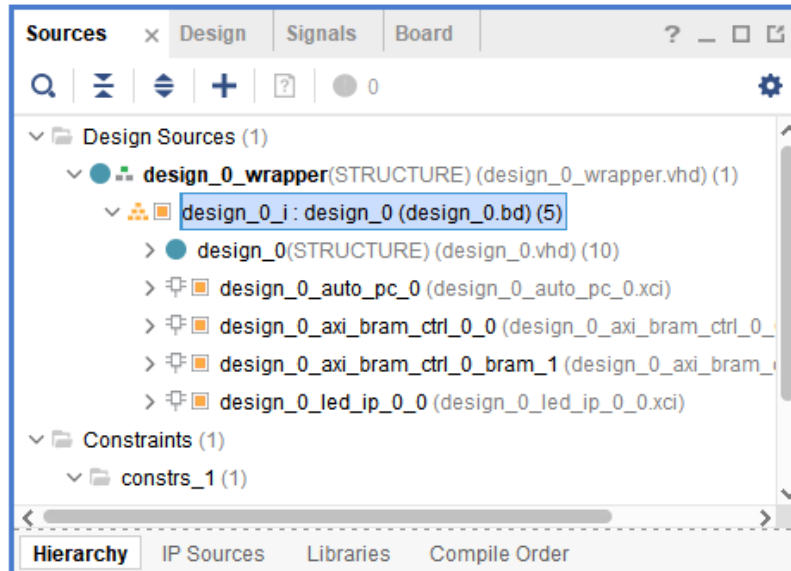
3-3. Add the provided lab3_*.xdc constraints file.

3-3-1. Download the `lab3_n4ddr.xdc` file from the source directory [{sources}](#)

3-3-2. Click **Add Sources** in the *Flow Navigator* pane, select **Add or Create Constraints**, and click **Next**.

3-3-3. Click the **Green Plus** button, and then **Add Files...**, select `lab3_n4ddr.xdc` for the Nexys4DDR board

- 3-3-4.** Click **Finish** to add the file.
- 3-3-5.** Expand Constraints folder in the *Sources* pane, and double click the **lab3_*.xdc** file entry to see its content. This file contains the pin locations and IO standards for the LEDs on the Nexys4DDR board. This information can usually be found in the manufacturer's datasheet for the board.
- 3-3-6.** Right click on *design_0.bd* and select *Generate output products*



- 3-3-7.** Click on **Generate Bitstream** and click **Yes** if prompted to save the Block Diagram, and click **Yes** again if prompted to launch synthesis and implementation. Click **Cancel** when prompted to *Open the Implemented Design*

After completing the implementation, save the results of the Design Runs window

By how much the resource consumption increased in relation to the project2?

Conclusion

Vivado IP packager was used to import a custom IP block into the IP library. The IP block was then added to the system. Connection automation was run where available to speed up the design of the system by allowing Vivado to automatically make connections between IP. An additional BRAM was added to the design. Finally, pin location constraints were added to the design.